
CS120 Project Report Fall 2024

Shunkang Jia
jiashk@shanghaitech.edu.cn

Yunfei Wang
wangyf9@shanghaitech.edu.cn

1 Introduction

We completed almost all mandatory tasks and none of the optional tasks. The project was implemented by Golang, JACK, ASIO4ALL. MATLAB and Wireshark are used for debugging.

2 Project 1: Acoustic Link

In Project 1, we finished all mandatory tasks and lost one point due to lack of accuracy (around 300 out of 10000 bits were wrong) in *Task 3: Transmitting Your First Bit*.

We completely followed algorithms in the provided `SamplePHY.m` and implemented one transmitter thread and one receiver thread. Detailed parameter is given below:

- Modulation: PSK; Carrier 10kHz sin wave
- 48 samples to represent 1 bit; 100 bits per frame
- Chirp as the preamble: 480 samples, range: 4kHz-12kHz-4kHz
- Frame structure: **Preamble**(480samples) + **Data**(100bits, 4800samples) + **CRC** (8bits)

Tune Power Threshold in Receiver Thread The biggest challenge in Project 1, in my opinion, is not algorithms or coding (simply rewrite the provided matlab script), but is tuning the threshold to determine where to align the preamble signal. Because the sound power is affected by a lot of things such as distance from the speaker, volume of the speaker, noise, etc.

Tune Parameter with MATLAB We utilize MATLAB in debugging and it makes things much easier than randomly tuning parameters. First, we stored samples output by transmitter and samples input to receiver in csv file. Second, we modify the provided matlab script to be consistent with our program. Finally, we analyze the csv file storing all samples and decode them with different parameters to see how many frames are detected.

High quality speaker It really drove me crazy when I found that the samples outputted by the transmitter are correctly decoded by matlab, but there is always 30% wrong when I tried to decode the input samples recorded the receiver. The problem is solved by simply changing the output speaker from my laptop's speaker to a headphone.

3 Project 2: Manage Multiple Access

We completed all mandatory tasks except the *Task4: CSMA with Interference*, because our implementation of CSMA mechanism is not effective.

Modulation We changed the modulation method according to one of the 2023 project reports. The modulation method is -1,-1,1,1 represents 1 and 1,1,-1,-1 represents 0. This modulation method is resistant to voltage difference between two computers.

MAC Frame Structure

- Preamble: Chirp Signal 48 Samples 2kHz-10kHz-2Khz
- PHYS Frame Header: length(9bit)
- MAC Frame Header: Dest(1bit) Src(1bit) Type(1bit) SlidingWindowID(4bit)
- MAC Frame Data (NULL if is ack): 400 bit
- CRC: 8 bit

Sliding Window Since our frame length is too small to utilize the full pipeline and speed up transfer, we decided to implement a sliding window for mac layer. We used MATLAB to analyze the input and output signal, and found out that approximately full pipeline contains 4 MAC frames with 400 bits. Then we implement the sliding window algorithm according to the slides in class.

CSMA Our implementation had some problems. I still introduce it here anyway. We set a constant power threshold to determine if there is a signal on audio cables, which is tuned in matlab. Then we took moving average of recent 64 samples comparing with the threshold. However, the two computers sometimes sense the signal at the same time and backoff simultaneously, and then restart simultaneously. To solve this issue, we apply Binary Exponential Backoff Algorithm in collision detection, meaning randomly picking a k from $\{0, 1, \dots, 2^n - 1\}$ where n is the number of backoff times, and backoff kT . However, the problem mentioned still exists and retransfer happens a lot.

4 Project 3: To the Internet

We finished all mandatory tasks in Project 3. The logic of implementing a router for sending ICMP is quite simple. However, figuring out which field to fill in and what value to fill in are the keys to the project.

Wireshark Wireshark is a tool to catch packet and users can also set filters such as `icmp` to monitor only ICMP packets. It helps a lot during debugging. For example, we initially manually defined a MAC address and did not compute checksum for IP packet. Although the packet is sent out, no response is returned, because the packet is considered invalid. With Wireshark, we compared our ICMP request packet with the valid ICMP packet sent by ping, and figure out the problem.

Define MTU in Aethernet Considering that IP packets are usually bigger than our Mac frame data size which only holds 400 bits or 50 bytes, we decided to use multiple mac frame to hold one IP packet. Since our sliding window size is 4, we decided to set an IP packet at maximum 4 MAC frame. We added 2 bits to mac frame header and added an IOHelper to buffer the not-yet-finished IP packet.

- 00: The first MAC frame of this IP packet.
- 11: The last MAC frame of this IP packet (i.e., can be passed to the upper layer).
- 01: This IP packet consists of only this single MAC frame (i.e., can be passed to the upper layer).
- 10: An intermediate frame within this IP packet.

Third Party Module: Gopacket Gopacket was helpful to us in Project 3. We used it to build packets and fill in different fields of different layers such as ICMP and IPv4. Moreover, we can also use Gopacket to serialize or deserialize the packet or byte stream to get the value of a specific field such as `srcIP` and `dstIP`.

5 Project 4: Above IP

We only implemented the DNS task in Project 4. We add dns layer and udp layer by module gopacket to sent ip packets, which is similar to what we have done with building icmp packets in Project 3.

6 Feedback

We have a basic bottom-up view of the computer networks after finishing the 4 projects and get used to tools such as wireshark. It is also pleasant to work on the project consistently in a semester instead of working on separate small projects.

Appendix: Solving JACK's Latency Problem

We did Project 1 and 2 relied on QJackCtl with the default driver chosen without the support of the ASIO4ALL driver. In project 3, I noticed an intolerable latency. I solved the problem eventually by enabling the ASIO4ALL driver on QJackCtl. It works with Win10 and Win11.

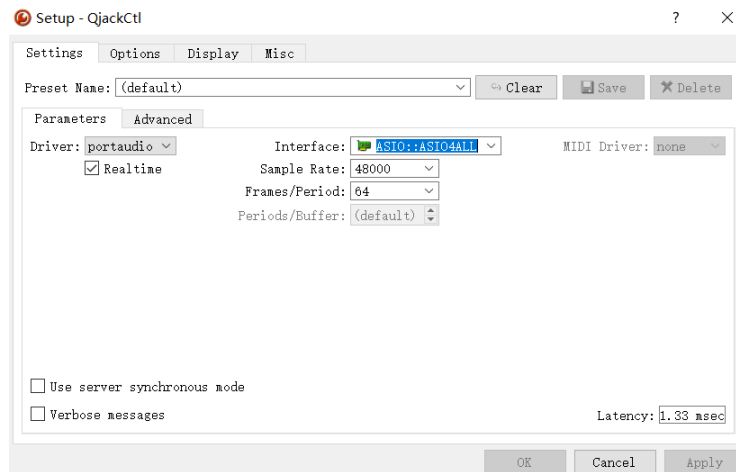


Figure 1: Example Setting

If directly clicking run from qjackctl fails, which often happens to me, you can alternatively choose to start jack and asio simultaneously in command line. Following is the command you can copy in a bat file and run.

```
@echo off
cd /d "C:\Program Files\JACK2"
.\jackd.exe -S -X winmm -dportaudio -d"ASIO::ASIO4ALL v2" -r48000 -p128
pause
```

References

- [1] Refer to jack starter in Golang: <https://github.com/wenxuanjun/jack-starter>
- [2] Project Document: <https://sist-cs120.github.io/project-doc/index.html>